

PYTHON

Module 12: Professional String Manipulation

CODING

String Fundamentals

At CodeHive, we define strings as 'Sequences of Unicode Characters'. They are the primary way we handle text in software.

Defining Strings: You can use single quotes (`'`) or double quotes (`"`). They are functionally identical.

Quotes Inside Quotes: To avoid errors, use different quotes for the wrapping and the content: `"It's a beautiful day"` or `'He said "Hello"'` .

Multiline Mastery: Use triple quotes (`"""` or `'''`) to store blocks of text that span multiple lines, preserving line breaks exactly as typed.

Strings as Arrays

Strings in Python are technically arrays of characters. This means you can access specific parts of a string using an index.

- Indexing:

`text[0]` gets the first character. Remember, Python starts counting at 0.

- Looping: Since strings are iterable, you can use a `for` loop to process one character at a time.

- Length: Use the `len()` function to calculate how many characters a string contains.

Slicing Techniques

Slicing allows you to extract a specific range from a string.

- Range Slicing: `b[2:5]` returns characters from index 2 to 4 (5 is excluded).

- From/To Start/End: `[:5]` slices from the beginning; `[2:]` slices to the very end.

- Negative Indexing: Use negative numbers to count from the end of the string. `b[-5:-2]` extracts relative to the final character.

String Modification Methods

Python provides built-in methods to transform strings. Note: Strings are **immutable**, so these methods return a **new** string rather than changing the original one.

- `upper()` / `lower()` : Changes the casing of the text.

- `strip()` : Removes leading and trailing whitespace—essential for cleaning user input.

- `replace("old", "new")` : Swaps specific substrings.

- `split(",")` : Breaks a string into a list based on a separator.

Concatenation & Search

Combining and searching within strings are daily developer tasks.

- Concatenation: Use the `+` operator to merge strings. Tip: Add a `" "` manually to include spaces.

- Membership Check: Use the `in` keyword to verify if a phrase exists within a string (e.g., `'free' in 'The best things are free'`).

- Negative Check: Use `not in` to verify the absence of text.

Modern Formatting (F-Strings)

F-Strings (formatted string literals) are the gold standard for combining variables and text.

Syntax: Place an `f` before the quotes and use `{}` placeholders.

Power Features:

- Expressions: `{20 * 5}` can be calculated inside the bracket.

- Modifiers: Use `:.2f` to format numbers to 2 decimal places.

Legacy Note: While `.format()` exists, f-strings are faster and more readable.

Escape Characters

To insert 'illegal' characters, use the backslash (\) escape symbol.

Common Escapes:

- \ " : Double Quote

- \ n : New Line

- \ t : Tab

- \\ : Literal Backslash

These allow for precise control over terminal and file output appearance.

Comprehensive Method Reference

Python includes dozens of methods. Key ones used at CodeHive include:

- `capitalize()` : Sentence case for the first letter.

- `isalpha()` / `isdigit()` : Validating that input contains only letters or numbers.

- `startswith()` / `endswith()` : Checking file extensions or protocol headers.

- `join()` : The efficient way to merge list items into a single string.